

1. UML Osztálydiagram részei

☐ **Használati eset diagram (Use Case Diagram):** A rendszer funkcióit (mit csinál a rendszer) mutatja be a felhasználók (Actorok) szemszögéből.

- **Kapcsolatok:** <<include>> (kötelezően magában foglal egy másik esetet), <<extend>> (adott feltétel esetén opcionálisan kiterjeszti azt).

☐ **Állapotgép diagram (State Machine Diagram):** Egy objektum lehetséges belső állapotait és az azok közötti átmeneteket írja le.

- **Esemény (Event):** Ami kiváltja az átmenetet (pl. gombnyomás, szignál).
- **Őrfeltétel (Guard):** Logikai feltétel, aminek teljesülnie kell az átmenethez (szögletes zárójelben, pl. [szabad_hely > 0]).
- **Tevékenység (Activity):** Amit az objektum az állapotba lépéskor, ott tartózkodáskor, vagy onnan kilépéskor csinál.
- **Tevékenységdiagram (Activity Diagram):** Főleg vezérlési folyamatokat ír le. Fő eleme a **Szinkronizációs sáv (Synchronization Bar / Fork-Join)**, ami a párhuzamos szálak szétválását és újraegyesülését (bevárását) jelöli.

Az UML (Unified Modeling Language) osztálydiagram a rendszer szerkezetét írja le.

- **Osztály (Class) doboz:** Három részre osztva: 1. Osztály neve, 2. Attribútumok (mezők/adattagok), 3. Operációk (metódusok).
- **Láthatósági jelek (Visibility):**
 - + Public (Mindenki látja)
 - - Private (Csak az osztály maga látja)
 - # Protected (Az osztály és a leszármazottai látják)
- **Kapcsolatok (Relationships):**
 - **Asszociáció (Association):** Sima vonal. Két osztály ismeri egymást.
 - **Aggregáció (Aggregation):** Üres rombusz. "Rész-egész" kapcsolat, de a rész önállóan is létezhet (pl. Egyetem és Professzor).
 - **Kompozíció (Composition):** Teli rombusz. Erős "rész-egész" kapcsolat. Ha az egész megsemmisül, a rész is (pl. Épület és Szoba).
 - **Öröklődés / Generalizáció (Generalization):** Folyamatos vonal, üres, zárt nyíllal. Szülő-gyermek kapcsolat.
 - **Megvalósítás / Realizáció (Realization):** Szaggatott vonal, üres, zárt nyíllal. Egy osztály implementál egy interfészt.
 - **Függőség (Dependency):** Szaggatott nyíl. Az egyik osztály működése függ egy másiktól (pl. paraméterként kapja).

- **Multiplicitás (Többszörösség):** Jelzi, hány példány vesz részt a kapcsolatban (pl. 1 - pontosan egy, 0..* - nulla vagy több, 1..* - legalább egy).

2. Tervezési minták (Design Patterns)

Gyakori szoftvertervezési problémákra adott, bevált "sablon" megoldások.

- **Singleton (Egyke):** Garantálja, hogy egy osztályból pontosan csak egyetlen példány létezzen a program futása során, és ehhez globális hozzáférést ad. Interfészek használata, öröklődési hierarchiában részt vesz.
- **Template Method (Sablonmetódus):** Egy algoritmus vázát (lépéseinek sorrendjét) rögzíti egy ősosztályban, és egy-egy konkrét lépés megvalósítását az absztrakt/virtuális metódusokon keresztül a leszármazottakra bízta.
- **Factory Method (Gyártófüggvény):** Az objektumpéldányosítást (a new kulcsszó használatát) kiszervezi egy külön metódusba, elrejtve a konkrét típusokat a kliens elől.
- **State (Állapot):** Lehetővé teszi, hogy az objektum viselkedése megváltozzon a belső állapotának változásakor (elkerülve a hatalmas if-else/switch blokkokat).
- **Strategy (Stratégia):** Algoritmusokat tokoz be osztályokba, és ezeket a kliens futásidőben szabadon cserélgetheti (pl. különböző rendezési algoritmusok). Különböző viselkedéseket külön osztályokba szervezi interfészeken keresztül.
- **Visitor (Látogató):** Új műveleteket adhatunk egy meglévő objektumstruktúrához anélkül, hogy az osztályokat módosítsuk (Double Dispatch technika).
- **Observer (Megfigyelő):** Egy "alany" állapotváltozásáról automatikusan értesíti a rá feliratkozott "megfigyelőket" (ezt valósítja meg a C# event/delegate rendszere).
- **Decorator (Díszítő):** Dinamikusan (futásidőben) új viselkedést vagy felelősséget ad hozzá egy meglévő objektumhoz anélkül, hogy alosztályok tömkelegét (pl. KaveTejjelCukorral) kellene létrehozni.
- **Composite (Összetétel):** Fa-szerkezetű (rész-egész) hierarchiák ábrázolása úgy, hogy a kliens kód számára teljesen mindegy legyen, hogy egyetlen apró elemmel, vagy egy összetett ággal dolgozik.
- **Proxy (Helyettes):** Egy "helyettesítő" vagy "képviselő" objektumot biztosít egy másik objektumhoz, hogy szabályozza a hozzá való hozzáférést.
- **Adapter (Illesztő / Burkoló):** Két inkompatibilis interfész összekötése. Egy meglévő (akár külső, megváltoztathatatlan) osztály interfészét olyanra "fordítja le", amelyet a kliens kód elvár.
- **Facade (Homlokzat):** Egy magas szintű, letisztult és egyszerűsített interfészt biztosít egy sok osztályból álló, bonyolult alrendszerhez. Elrejtja a káoszt a kliens elől.
- **Bridge(Híd):** Szétválasztja egy objektum absztrakcióját és implementációját úgy, hogy a kettő egymástól függetlenül (külön ágon) variálható és kiterjeszthető legyen.
- **Iterator(Bejáró):** Gondoskodik arról, hogy egy aggregált objektum (pl. egy fa vagy egy lista) elemeit szekvenciálisan végig tudjuk látogatni anélkül, hogy feltárnánk annak bonyolult belső szerkezetét. Az Iterator szétválasztja az adattárolást és a bejárás módját. A kliens csak a

Next() és a HasNext() metódusokat ismeri, nem kell tudnia, hogy mögötte láncolt lista, tömb vagy bináris fa áll.

- **Command(parancs):** Egy kérést (cselekvést) önálló objektummá alakít (encapsulate request as an object). Ez lehetővé teszi a kérések paraméterezését, várakozási sorba tételét (Queue), naplózását, és a legfontosabb: a **Visszavonást (Undo/Redo/Execution)**.
- **Mediator (Közvetítő):** Csökkenti az objektumok közötti közvetlen, kaotikus kapcsolatokat (hogya ne mindenki mindenkire ismerjen -> tight coupling). A kommunikációt egy központi objektumba (Mediátorba) tereli át.
- **Chain of Responsibility (Felelősséglánc):** Elkerüli a kérés küldője és fogadója közötti szoros csatolást azáltal, hogy a kérést egy objektumokból álló láncon (chain) küldi végig. (láncszemek)
- **Builder(építő):** Egy nagyon bonyolult objektum felépítését választja szét a reprezentációjától. Így ugyanaz a felépítési folyamat különböző reprezentációkat is létrehozhat. Lehetővé teszi, hogy egy objektumot lépésről lépésre építsünk fel (elkerülve a 10-paraméteres "szörny-konstruktorokat").
- **Prototype (Prototípus):** Új objektumok példányosítása (a drága new hívása helyett) úgy, hogy egy már meglévő, felkonfigurált objektumot (a prototípust) másolunk/klónozzunk le. C#-ban ehhez gyakran az `ICloneable` interfészt használják.
- **Bridge (Híd):** Szétválaszt egy absztrakciót a megvalósításától (implementációjától), hogy a kettő egymástól függetlenül változhasson. Megelőzi az öröklődési fa "robbanását" (amikor minden kombinációra új osztályt kéne írni).
- **Memento (Emlékeztető / Emlék):** Megmenti (és később visszaállítja) egy objektum belső állapotát anélkül, hogy megsértené az egységbezárás (Encapsulation / adatrejtés) elvét.

2. C# Specifikus tudások

- **ref vs out:** A ref paramétert kötelező inicializálni a metódushívás előtt, míg az out paraméternél ez nem feltétel, viszont a metóduson belül kötelező értéket adni neki. A ref a már létező változók referenciáját adja át, az out pedig kifejezetten arra szolgál, hogy a metódus több visszatérési értéket tudjon 'kibocsátani' az inicializálatlan változókba.
- **record:** Egy olyan referenciatípust definiál, amely beépítetten támogatja az érték alapú egyenlőségvizsgálatot a kódunkban. Automatikusan implementálják az `IComparable` interfészt. Tartalmuk alapján hasonlíthatók össze, nem memóriacím alapján.
- **Class vs. Struct:** A class referencia típus (Heap memóriában tárolódik), a struct érték típus (Stack memóriában tárolódik, másolódik átadáskor).

- **Felsoroló minta (IEnumerator és IEnumerable):** Az IEnumerable jelzi, hogy egy gyűjtemény bejárható (pl. foreach ciklussal). Az iterációt a háttérben az IEnumerator végzi a MoveNext() metódussal és a Current tulajdonsággal.
- **Properties (Tulajdonságok):** get és set blokkokkal rejtik el a belső mezőket (Enkapszuláció), így ellenőrizhetjük, mit és hogyan állítanak be.
- **Virtual és Override:** A polimorfizmus alapja. Az őssztályban a virtual engedélyt ad a metódus módosítására, a leszármazottban az override cseréli le a viselkedést.
- **is és as operátorok:** Az is logikai (bool) vizsgálatot végez, hogy egy objektum adott típusú-e. Az as biztonságosan (kivételdobás nélkül) kényszeríti (castolja) az objektumot a megadott típusra; ha nem sikerül, null értéket ad.
- **Delegate és Event:** A delegate egy típusbiztos mutató egy vagy több metódusra. Az event egy biztonságos burkoló, ami megakadályozza, hogy kívülről jogosulatlanul felülírják a feliratkozókat vagy elsüssék az eseményt.
- **const vs. readonly:** A const egy fordítási időben (compile-time) kiértékelt, szigorú állandó. A readonly egy futásidejű (run-time) állandó, aminek az értékét az osztály konstruktorában dinamikusan is be lehet állítani, de utána már nem módosítható.
- **Kiterjesztő metódusok (Extension Methods):** Meglévő osztályok (akár a beépített string vagy List) bővítése új metódusokkal származtatás nélkül. (Statikus osztályban, statikus metódusként írjuk, az első paraméter elé a this kulcsszót tesszük).
- **Yield Return:** Lehetővé teszi az egyedi iterátorok (felsorolók) írását és a "lusta kiértékelést" (lazy evaluation), ami azt jelenti, hogy az elemek csak akkor jönnek létre/számolódnak ki, amikor a foreach ciklus ténylegesen kéri őket. C#-ban: Generál egy rejtett állapotgépet (State Machine), amely eltárolja a helyi változókat a hívások között.
- **Kivételkezelés (try-catch-finally):** A throw; újra eldobja a hibát megtartva a Call Stack-et (hívási vermet), míg a throw ex; lenullázza azt (ez rossz gyakorlat). A finally blokk garantáltan lefut (pl. fájlok bezárására jó).
- **Generics (<T>):** A List<T>-hez hasonló típusoknál biztosítja a fordítási idejű típusbiztonságot és elkerüli a lassú dobozolást (Boxing/Unboxing).

4. SOLID Elvek

Öt alapvető elv a tiszta, karbantartható objektumorientált kódért.

- **S - Single Responsibility (Egyetlen felelősség elve):** Egy osztálynak csak egyetlen feladata legyen, azaz csak egyetlen oka lehessen a változtatásra.
- **O - Open/Closed (Nyílt/Zárt elv):** A kód legyen nyitott a kiterjesztésre (új funkciók hozzáadása), de zárt a módosításra (a meglévő, letesztelt kód átírása).
- **L - Liskov Substitution (Liskov helyettesítési elv):** A leszármazott osztályok maradéktalanul, hiba vagy a program logikájának megborulása nélkül helyettesíteni tudják az őssztályukat (pl. ne dobjanak NotImplementedException-t egy kötelező ős-metódusra).

- **I - Interface Segregation (Interfész elválasztás elve):** Több, apró és specifikus interfész sokkal jobb, mint egy hatalmas, "mindent tudó" (fat) interfész. Senkit ne kényszerítsünk olyan metódus megvalósítására, amire nincs szüksége.
- **D - Dependency Inversion (Függőség megfordítása elve):** A magas szintű modulok (üzleti logika) ne függjenek az alacsony szintű moduloktól (pl. konkrét adatbázis hívások). Mindkettő absztrakcióktól (interfészekről) függjön. Erre jó a Dependency Injection (konstruktor injektálás).

5. Adatszerkezetek (C# gyűjtemények)

- **List:** Dinamikus tömb. Akkor használd, ha index alapján akarsz elemeket elérni és a sorrend számít.
- **Stack (Verem): LIFO (Last-In, First-Out).** Az utoljára betett elemet veszed ki először (pl. Visszavonás/Undo funkció). Metódusai: Push() és Pop().
- **Queue (Sor): FIFO (First-In, First-Out).** Aki először beállt, az jön ki először (pl. Nyomtató várólista). Metódusai: Enqueue() és Dequeue().
- **HashSet (Halmaz):** Csak egyedi elemeket tárol (nincs duplikáció). A keresés (Contains) benne rendkívül gyors ($O(1)$), de az elemek nem indexeltek.
- **Dictionary<TKey, TValue> (Szótár):** Kulcs-érték párokat tárol. A kulcsnak egyedinek kell lennie. Rendkívül gyorsan kikeresi az értéket a kulcs alapján (értelmes a TryGetValue-t használni a biztonságos kereséshez).
- **abstract vs virtual vs new:** * abstract: Nincs törzse, *kötelező* felülírni a leszármazottban.
- virtual: Van alapértelmezett törzse, *lehet* (opcionális) felülírni (override).
- new (metódus elé írva): Név-elrejtés (Name Hiding). Megszakítja a polimorfizmust, nem írja felül az őst, csak eltakarja.
- **IDisposable és a using blokk:** A nem menedzselt erőforrások (pl. fájlok, hálózati kapcsolatok) determinisztikus, azonnali felszabadítására való, amit a Garbage Collector (GC) nem tud garantálni. A using egy rejtett try-finally blokk, ami a végén meghívja a Dispose()-t.
- **LINQ és Késleltetett kiértékelés (Deferred Execution):** A Where és Select metódusok csak deklarálják a lekérdezést. A futás csak akkor történik meg, amikor ténylegesen bejárjuk (pl. foreach vagy ToList()).
- **Action és Func delegáltak:** A Func mindig ad vissza értéket (utolsó paraméter a visszatérési típus), az Action sosem (visszatérési típusa void).
- **Többszálúság (TPL - Task Parallel Library):** Az async / await páros nem blokkolja a hívó szálát. A Task egy intelligens feladat, a Thread egy nehézkes operációs rendszer szintű szál. A lock a versenyhelyzetek (Race condition) elkerülésére való (háttérben a Monitor osztályt használja).
- **Tuple vs. ValueTuple:** A ValueTuple egy memóriahatékony, veremben (stack) tárolt struktúra (struct), ami több visszatérési érték visszaadására szolgál.
- **Tesztelés (Egységtesztelés / Unit Testing) – Ez teljesen kimaradt, pedig fontos!**
- **AAA minta:** Arrange (Előkészítés), Act (Cselekvés/Hívás), Assert (Ellenőrzés).

- **MSTest Attribútumok:** [TestInitialize] (minden teszt előtt lefut), [ClassCleanup] (az összes teszt után fut le egyszer).
- **Mock vs. Stub:** A *Stub* csak "buta" konzerv-válaszokat ad a teszthez. A *Mock* okosabb: elvárásokat tárol (pl. "ezt a metódust hívják-e meg"), és a viselkedést ellenőrzi.

Árulkodó jelek és kulcsszavak a tesztkérdésekben (A "Puska")

A feleletválasztós vizsgákon a kérdés megfogalmazásában gyakran ott van a válasz. Ha ezeket a kulcsszavakat látod, szinte biztosan az adott elvre vagy mintára kell gondolni:

Tervezési minták felismerése:

- *Kulcsszó:* "Visszavonás", "Undo/Redo", "Cselekvés sorba állítása" → **Command (Parancs)** (vagy Memento, de a Command a cselekvést tárolja).
- *Kulcsszó:* "Algoritmus cseréje futásidőben", "Kliens választja ki" → **Strategy (Stratégia)**.
- *Kulcsszó:* "Hatalmas if-else/switch kiváltása", "Belső állapottól függő viselkedés" → **State (Állapot)**.
- *Kulcsszó:* "Fa szerkezet", "Rész és egész egységes kezelése" → **Composite (Összetétel)**.
- *Kulcsszó:* "Inkompatibilis", "Fordít/Lefordít", "Meglévő kód illesztése" → **Adapter**.
- *Kulcsszó:* "Bonyolult alrendszer", "Egyszerűsített belépési pont/interfész" → **Facade (Homlokzat)**.
- *Kulcsszó:* "Káosz csökkentése", "Sok-a-sokhoz kapcsolat elvágása", "Központi irányító" → **Mediator (Közvetítő)**.
- *Kulcsszó:* "Új művelet hozzáadása osztálymódosítás nélkül", "Double dispatch" → **Visitor (Látogató)**.
- *Kulcsszó:* "Memóriamegtakarítás", "Közös tulajdonságok megosztása", "Sok ezer apró objektum" → **Flyweight (Pehelysúlyú)**.
- *Kulcsszó:* "Absztrakció és implementáció szétválasztása", "Osztály-robbanás elkerülése" → **Bridge (Híd)**.

SOLID elvek felismerése:

- *Kulcsszó:* "Egyetlen oka van a változásra" → **SRP (Single Responsibility)**.
- *Kulcsszó:* "Bővíthető, de nem módosítható" → **OCP (Open/Closed)**.
- *Kulcsszó:* "Ősosztály helyettesíthetősége", "Szerződés", "Előfeltétel/Utófeltétel" → **LSP (Liskov)**.
- *Kulcsszó:* "Kövér/Mindent tudó interfészek", "Felesleges metódusok implementálása" → **ISP (Interface Segregation)**.

- *Kulcsszó*: "Absztrakcióktól függ", "Magas szintű nem függ az alacsonytól" → **DIP (Dependency Inversion)**.

C# Nyelvi trükkök és csapdák:

- **Fordítási idő (Compile-time) vs. Futásidő (Run-time)**: Erre nagyon figyelj! A `const`, az `readonly` beállítása, az `override` (késői kötés/polimorfizmus), és az `is/as` vizsgálat *futásidőben* történik.
- **A "Mindig", "Kizárólag", "Soha" szavak**: Ha egy válaszlehetőségben illet látsz (pl. "A struct *kizárólag* int-et tárolhat", "Az interface *soha* nem lehet..."), az az esetek 90%-ában **hamis** állítás.
- **Adatszerkezeteknél**: * Ha a gyors keresés/egyediség a kérdés ($O(1)$ komplexitás) → **HashSet** vagy **Dictionary**.
 - Ha LIFO (legutolsó jön ki először, pl. visszavonás) → **Stack**.
 - Ha FIFO (első jön ki először, pl. várakozás) → **Queue**.
- **is vs. as**: Ha a kérdésben "logikai igaz/hamis" szerepel → `is`. Ha "konvertál és null-t ad hiba esetén" → `as`.